

Tema 02. Regresión y clasificación

(Parte 2: Regresión multivariable)

Autor: Ismael Sagredo Olivenza

2.2. Regresión lineal con múltiples variables

Hasta ahora hemos visto la regresión para una variable, pero en machine learning lo normal es crear modelos que necesiten múltiples variables (y cada vez los modelos son más grandes)

Size in feet ²	Number of bedrooms	Number of floors	Age of home (years)	Price (\$) in 1000's
X ₁	X ₂	X ₃	X ₄	
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

2.2.1 ¿Cómo generalizamos la ecuación?

Una variable es $f(w, b) = w * x + b$, para múltiples variables:

$$f(w, b) = w_1 * x_1 + w_2 * x_2 + \dots w_n x_n + b$$

Si nos fijamos en los datos anteriores vemos que:

x_1 = Size in feet

x_2 = Number of bedrooms

x_3 = Number of floors

x_4 = Age of home

$$f(w_1, w_2, w_3, w_4, b) = 0.1 * x_1 + 4 * x_2 + 10x_3 - 2x_4 + 80$$

2.2.1.1 Vectorizamos

$[w_1, w_2, \dots, w_n]$ el vector de w's son los parámetros del modelo y **b** es un número.

$$f_{\vec{w},b}(\vec{x}) = \vec{w} * \vec{x} + b$$

Detalles de implementación en Numpy

```
w = np.array([1.0, 2.5, -3.3])
b = 4
x = np.array([10, 20, 30])

f = np.dot(x, w) + b # producto escalar
## Without vectorization for j in range(0, n)
```

Recordemos que es el producto escalar (Dot Product)

El producto escalar entre dos vectores $\vec{u} = (u_x, u_y)$ y $\vec{v} = (v_x, v_y)$ es:

$$\vec{u} \cdot \vec{v} = u_x \cdot v_x + u_y \cdot v_y$$

Ejemplo:

$$\vec{u} = (1, 2) \quad \vec{v} = (-1, 3)$$

$$\vec{u} \cdot \vec{v} = (1, 2) \cdot (-1, 3)$$

$$= 1 \cdot (-1) + 2 \cdot 3$$

$$= -1 + 6 = 5$$

Ejemplos en Python:

```
import numpy as np
u = np.array((1,2))
v = np.array((2,3))
print(np.dot(u,v))
# 28
a = [[1, 0], [0, 1]]
b = [[4, 1], [2, 2]]
print(np.dot(a,b))
# [[4 1]
# [2 2]]
```

Dot product para matrices:

$$\begin{bmatrix} u_1 & u_2 \\ u_3 & u_4 \end{bmatrix} \cdot \begin{bmatrix} v_1 & v_2 \\ v_3 & v_4 \end{bmatrix} = \begin{bmatrix} u_1 * v_1 + u_2 * v_3 & u_1 * v_2 + u_2 * v_4 \\ v_3 * v_1 + u_4 * v_3 & u_3 * v_2 + u_4 * v_4 \end{bmatrix}$$

Dot Product

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 7 & 8 \\ 9 & 10 \\ 11 & 12 \end{bmatrix} = \begin{bmatrix} 58 \end{bmatrix}$$

Numpy dot product puede utilizar una implementación optimizada obtenida como parte de "BLAS" (the Basic Linear Algebra Subroutines)

* https://es.wikipedia.org/wiki/Basic_Linear_Algebra_Subprograms

Esta biblioteca obtiene ventaja del hardware (si existe), como instrucciones SIMD (Simple Instruction, Multiple Data), caché, multinúcleo, etc. Si es posible, es preferible por razones de eficiencia describir las operaciones en forma vectorial

- SISD: Simple Instruction, Simple Data (escalar)
- SIMD: Simple Instruction, Multiple Data (3Dnow!, MMX/SSE)
- MIMD: Multiple instruction, Multiple Data (multicore vectorial)

2.2.1.2 Ecuaciones de Gradient descent.

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^m (y' - y_i) x_{j,i} \forall j = 1..n$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (y_i' - y_i)$$

$$\vec{y}' = F_{\vec{w},b}(\vec{x})$$

Repetir para todo n (numero de pesos)

$$w_1 = w_1 - \alpha \frac{1}{m} \sum_{i=1}^m (y_1' - y_i) x_{1,i} \dots$$

$$w_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^m (y_n' - y_n) x_{n,i}$$

Algunas cuestiones:

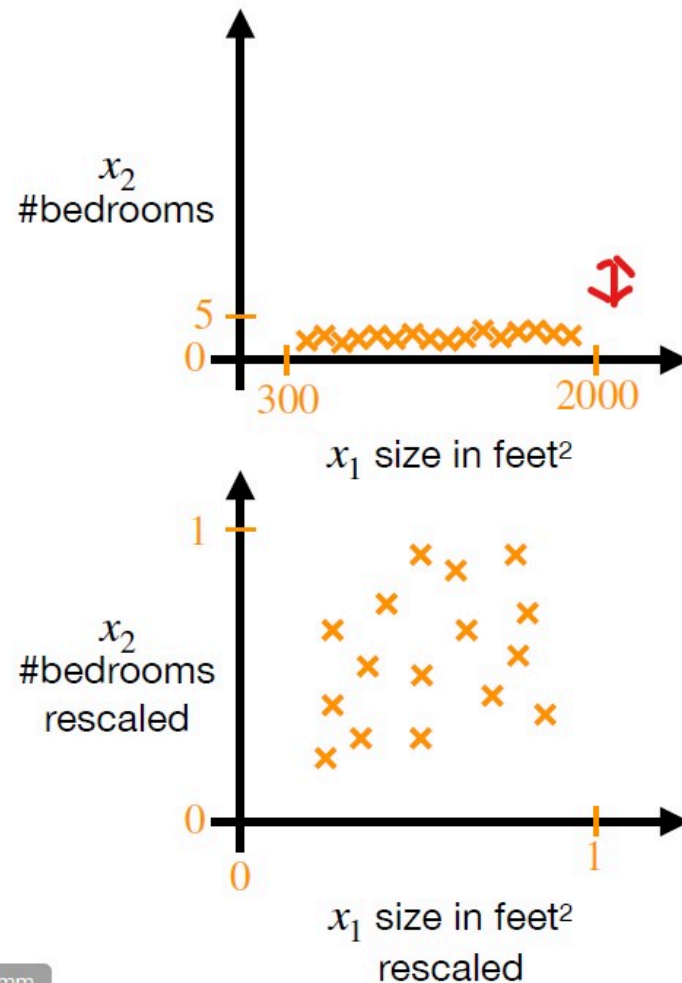
- En vectorial es utilizar las operaciones de calculo vectorial. Restar dos vectores se hace automaticamente con Numpy. Lo mismo que sumar o multiplicar. La combinación de productos y sumas es el dot product.
- En interativo es hacer un for que sustituya al sumatorio.
- Preservar los valores de **b** y de los **ws** para el cálculo y sustituirlos al final.

2.2.2 Escalado (Feature Scaling)

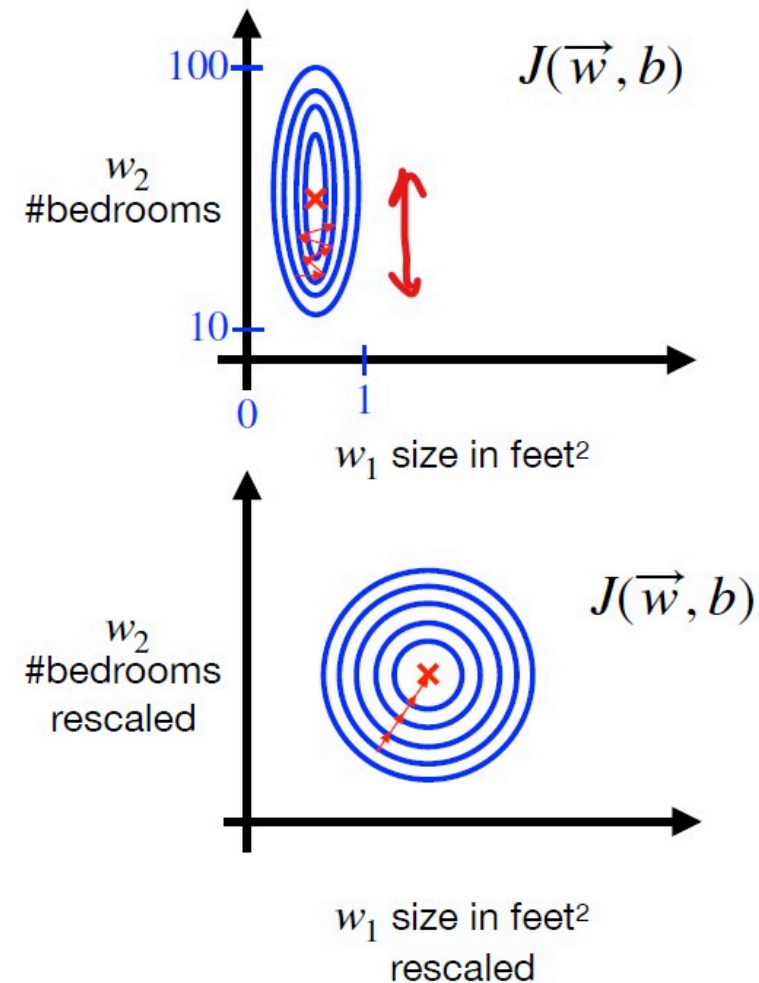
- Las características deben estar en la misma escala.
- ¿Cuál debe ser la escala de los parámetros W ?
- Como el tamaño del error modifica los parámetros mediante el descenso de gradiente, tener diferentes escalas hace que algunos parámetros se ajusten mejor que otros.
- Reescalando todos los valores a una escala similar, los parámetros se modifican con la misma proporción.
- Unos saltos demasiado grandes o demasiado pequeños (como ocurre con la tasa de aprendizaje) dificultarán encontrar el peso correcto que minimice la función.

Feature size and gradient descent

Features



Parameters

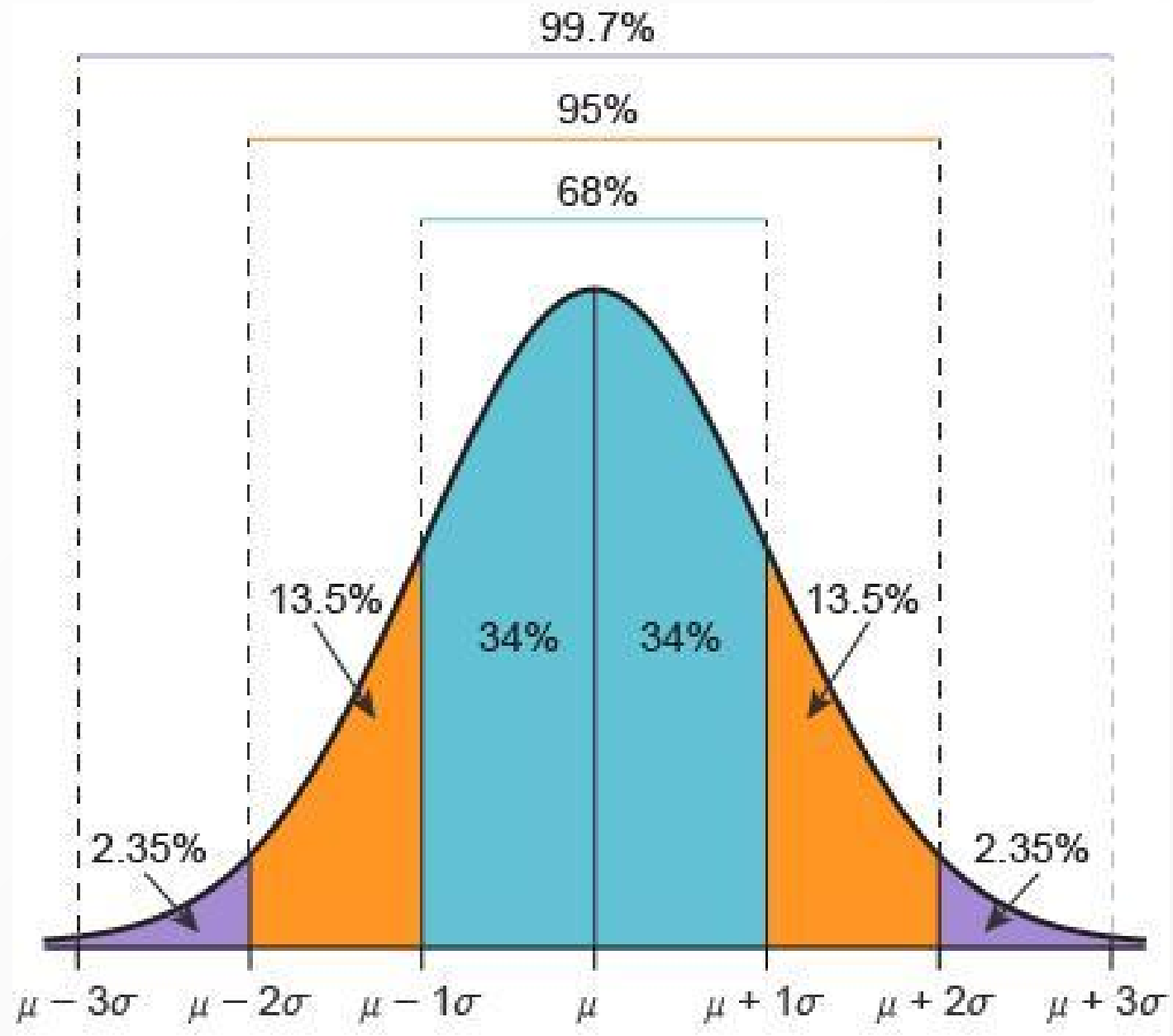


En el ejemplo anterior, el dominio de X_1 va de 300 a 2000 $\Rightarrow$ 1700 y x_2 va de 0 a 5 (tamaño 5):

Podemos transformar fecha dividiendo por max (si podemos ajustar el min a 0, podemos restar min/max). Lo que buscamos es que ambas variables se muevan entre los valores 0 a 1.

También podemos normalizar restando la media y dividiendo por el rango (o MAX-MIN) que mantiene los valores entre -1 y 1

Finalmente podemos normalizar usando **Z-Score** (restando por la media y dividiendo por la desviación típica). Z-Score: las características tienen media 0 y desviación 1 pero puede haber valores fuera del rango -1 a 1 (en los extremos de la campana)



2.2.3 ¿Cómo establecer el criterio de convergencia?

Podemos establecer diferentes criterios de convergencia:

- Que el error sea menor que uno previamente determinado
- Que no se produzca una mejora significativa en el rendimiento del modelo tras n iteraciones de descenso gradiente
- El el modelo pare cuando acabe el número de iteraciones máximo **(Siempre tiene que haber un número máximo de iteraciones para garantizar la parada si no se complen los otros criterios)**

2.2.4 Convergencia: elección del learning rate

- Normalmente, es buena heurística elegir **learning rates** bajos porque se evitan saltos.
- Si el error de entrenamiento comienza a subir, o tiene muchas oscilaciones, también es un síntoma de que el learning rate es muy alto
- Si es demasiado bajo, el numero de iteraciones para converger aumenta y tambien puede quedarse en minimo local (esto es más complicado de detectar)
- El learning rate puede variar con el tiempo para simular distintas fases de exploracion y explotación.